ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO MÔN: NGÔN NGỮ LẬP TRÌNH C#

Lớp: DCT123C4
Nhóm: 18
Sinh viên thực hiện:

3123411230 – NGUYỄN LÊ TẤN PHÚC
3123411238 – VŨ HOÀNG PHƯƠNG

Giảng viên hướng dẫn: NGUYỄN QUỐC HUY

Thành phố Hồ Chí Minh, 2026

TÀI LIỆU YÊU CẦU SẢN PHẨM (PRD) - HỆ THỐNG THUYẾT MINH ĐA NGÔN NGỮ PHỐ ẨM THỰC VĨNH KHÁNH 3
1. Tổng quan dự án (Project Overview) 3
2. Đối tượng người dùng (User Personas) 3
3. Kiến trúc kỹ thuật tổng quan (System Architecture) 4
4. Yêu cầu tính năng chi tiết (Functional Requirements) 4
4.1. Module Xác thực & Quản lý người dùng (Authentication & User Management) 4
4.2. Module Quản lý Địa điểm ẩm thực (POI Management) 4
4.3. Module Đa ngôn ngữ (Localization & Hydration) 5
4.4. Module Thống kê lượt quan tâm (Visit Tracking & Analytics) .. 6
4.5. Module Hệ thống Email tự động (Notification/Email Service) .. 6
4.6. Module Tích hợp Bản đồ & Vị trí (Map Integration) 6
4.7. Module Mã QR & Thuyết minh tự động (QR & TTS) 6
4.8. Module Hàng rào địa lý (Geofencing) 7
5. Quy ước Trạng thái Hệ thống (State Machine) 7
5.1. Trạng thái của Địa điểm (POI Status) 7
6. Lộ trình cải tiến tương lai (Future Roadmap) 7
7. Yêu cầu Phi chức năng (Non-Functional Requirements) 8
8. Luồng Mobile App (Mobile Application Flow) 8
9. Checklist Kiểm thử (Quality Assurance Checklist) 8
10. Quy tắc Dữ liệu & Ràng buộc Logic (Data Rules & Constraints) 9
11. Thiết kế Luồng Dữ liệu (API Data Flow) 9
12. Kế hoạch Triển khai & Cấu hình (Deployment & Configuration) 9
13. Ma trận Phê duyệt & Email (Approval & Email Matrix) 10
14. Quản lý Lỗi & Phản hồi Hệ thống (Error Handling & System Feedback) 10
15. Chỉ số hiệu suất chính (Key Performance Indicators - KPIs) 11
16. Nguyên tắc Thiết kế Giao diện (UX/UI Design Principles) 11
17. Sequence Diagram 11
17.1. Khởi động và Kiểm tra kết nối (Startu1. Đăng nhập và Kiểm tra quyền (Authentication) 11
17.2. Tải danh sách địa điểm - Thành công (Load Data Success) ... 13
17.3. Dashboard Tổng quát (Admin) 14
17.4. Phê duyệt hoặc Từ chối POI (Admin) 15
17.5. Tạo Địa điểm ẩm thực mới (Owner) 16
17.6. Xóa mềm và Khôi phục POI (Owner/Admin) 17
17.7. Cập nhật thông tin & Đồng bộ Ngôn ngữ (Owner) 18
17.8.Tìm kiếm quán ăn xung quanh (Mobile User) 19

17.9. Luồng Quét mã QR (QR Code Scanning Flow) 19
17.10. Luồng Thuyết minh tự động (TTS Narration Flow) 21
17.11. Hàng rào địa lý - Geofence (Mobile User) 22
17.12. Gửi Email thông báo sự kiện (Backend) 23
17.13. Luồng Đăng ký tài khoản (Registration) 24
17.14. Luồng Tải lên & Hiển thị hình ảnh (Image Upload) 25
17.15. Luồng Phân quyền Dashboard (Data Isolation) 26
18. Kế hoạch Bảo trì & Hỗ trợ (Maintenance & Support) 27

TÀI LIỆU YÊU CẦU SẢN PHẨM (PRD) - HỆ THỐNG THUYẾT MINH ĐA NGÔN NGỮ PHỐ ẨM THỰC VĨNH KHÁNH

1. Tổng quan dự án (Project Overview)

Hệ thống thuyết minh đa ngôn ngữ phố ẩm thực Vĩnh Khánh là một hệ sinh thái ứng dụng quản lý và khám phá địa điểm ẩm thực (Point of Interest - POI). Hệ thống kết nối ba nền tảng chính:

- Backend (API): Trái tim của hệ thống, xử lý logic, lưu trữ dữ liệu tập trung.
- Web Portal (Admin/Owner UI): Nơi Quản trị viên (Admin) quản lý nền tảng và Chủ quán (Owner) quản lý cửa hàng của mình.
- Mobile App (Client App): Ứng dụng dành cho người dùng cuối (thực khách) để tìm kiếm, khám phá các địa điểm ẩm thực.

2. Đối tượng người dùng (User Personas)

Hệ thống phân chia thành 3 nhóm người dùng chính với các quyền hạn riêng biệt:

Quản trị viên (Admin): Người điều hành nền tảng. Quản lý tài khoản người dùng, kiểm duyệt (Duyệt/Từ chối) các POI do Chủ quán đăng lên, theo dõi thống kê toàn hệ thống.

Chủ quán (Owner): Người sở hữu các địa điểm ẩm thực. Đăng tải thông tin quán (POI), chỉnh sửa thông tin, xem thống kê lượng quan tâm (lượt truy cập/view) riêng cho quán của mình.

Người dùng cuối (End-user): Sử dụng Mobile App để xem danh sách quán, tìm quán gần đây. Mọi thao tác xem chi tiết quán sẽ được hệ thống ghi nhận thành "Lượt quan tâm" (Visit).

3. Kiến trúc kỹ thuật tổng quan (System Architecture)

Hệ thống được xây dựng theo mô hình Monorepo với các công nghệ:

- Backend: ASP.NET Core Web API, cơ sở dữ liệu MongoDB (CulinaryDB). Cấu hình CORS mở, áp dụng Dependency Injection cho các Services (PoiService, EmailService, PoiVisitService). Cung cấp Swagger để test API.
- Frontend (Portal): Blazor WebAssembly/Server, sử dụng thư viện UI MudBlazor.
- Mobile: .NET MAUI (tiêu thụ API từ Backend).

4. Yêu cầu tính năng chi tiết (Functional Requirements)

4.1. Module Xác thực & Quản lý người dùng (Authentication & User Management)

Đăng ký/Đăng nhập:

- Hỗ trợ tạo tài khoản mới và đăng nhập bằng Username/Password (mật khẩu được mã hóa BCrypt).
- Hệ thống từ chối đăng nhập đối với các tài khoản đang bị khóa (IsActive = false).

Quản lý người dùng (Dành cho Admin):

- Xem danh sách toàn bộ người dùng.
- Cập nhật thông tin: Phân quyền (Role), Tên đầy đủ, Gắn ID Chủ quán (OwnerId).
- Thao tác nhanh: Khóa/Mở khóa tài khoản (Toggle-active) và Xóa người dùng.
- Tự động hóa: Khi Admin Khóa/Mở khóa hoặc Xóa người dùng, hệ thống tự động gửi Email thông báo cho người dùng đó.

4.2. Module Quản lý Địa điểm ẩm thực (POI Management)

Luồng dữ liệu của POI áp dụng cơ chế "Soft Delete" (Xóa mềm). POI bị xóa sẽ không mất khỏi Database mà chuyển trạng thái sang deleted.

A. Quyền hạn của Chủ quán (Owner UI - My POIs):

Tạo mới POI: Owner tạo POI bắt buộc phải gắn OwnerId. POI mới tạo mặc định ở trạng thái pending (Chờ duyệt).

Quản lý POI cá nhân:

- Xem danh sách các quán của mình (Bao gồm cả các quán đã bị xóa mềm).
- Cập nhật thông tin quán (Tên, Ảnh, Vị trí, Dữ liệu đa ngôn ngữ).
- Xóa quán của mình (Chuyển thành deleted) và Khôi phục quán đã xóa (Chuyển lại thành pending).
- Click vào chi tiết quán (Hành động này đồng thời kích hoạt tracking lượt visit).

B. Quyền hạn của Quản trị viên (Admin UI - POI Manager):

Lưu ý: Admin không được phép tạo mới hay chỉnh sửa trực tiếp nội dung POI.

Kiểm duyệt POI:

- Xem chi tiết thông tin và vị trí (Map) của các POI.
- Duyệt (Approve) hoặc Từ chối (Reject) các POI đang ở trạng thái chờ.

Quản lý trạng thái:

- Có thể Xóa mềm (Delete) hoặc Khôi phục (Restore) các POI.
- Xem được toàn bộ POI bao gồm cả những POI đã xóa (Giao diện hiển thị nhãn "Đã xóa").

4.3. Module Đa ngôn ngữ (Localization & Hydration)

Dữ liệu POI hỗ trợ đa ngôn ngữ (Lưu trong Localizations).

Cơ chế Fallback: Khi client yêu cầu một ngôn ngữ (VD: Tiếng Việt vi), nếu không có dữ liệu, hệ thống tự động fallback sang Tiếng Anh (en), nếu vẫn không có sẽ trả về dữ liệu gốc. Nếu thiếu Tiêu đề (Title), trả về chuỗi rỗng.

Hệ thống gồm 5 ngôn ngữ: Tiếng Việt, Tiếng Anh, Tiếng Hàn, Tiếng Nhật và Tiếng Trung.

4.4. Module Thống kê lượt quan tâm (Visit Tracking & Analytics)

Ghi nhận lượt xem: Mỗi khi Mobile App (hoặc Owner) mở xem chi tiết một POI, hệ thống ghi nhận 1 record vào collection PoiVisits.

- Dashboard Admin: * Hiển thị KPI tổng quan (Số lượng POI, Admin, Owner, Tài khoản bị khóa).

Biểu đồ phân bổ danh mục quán ăn.

Bảng xếp hạng Top các quán được quan tâm nhất (Lấy từ aggregated stats).

- Dashboard Owner:

Hiển thị KPI và chỉ số lượt quan tâm (visits) chỉ dành riêng cho các POI thuộc sở hữu của Owner đó.

Chỉ hiển thị dữ liệu của các POI đang hoạt động (ẩn các POI đã xóa).

4.5. Module Hệ thống Email tự động (Notification/Email Service)

Hệ thống tự động gửi email dựa trên các sự kiện (Events). (Lưu ý: Chỉ gửi thành công nếu Username của người dùng định dạng là Email hợp lệ).

Sự kiện POI: Gửi mail cho Owner khi POI được Tạo mới, Cập nhật, Xóa, Khôi phục, Được duyệt, hoặc Bị từ chối.

Sự kiện Tài khoản: Gửi mail cho User khi tài khoản bị Khóa, Mở khóa, hoặc Xóa.

4.6. Module Tích hợp Bản đồ & Vị trí (Map Integration)

Định vị POI: Mỗi POI lưu trữ tọa độ địa lý (Latitude, Longitude) để hiển thị vị trí chính xác trên bản đồ.

Giao diện Map: * Admin/Owner: Hiển thị bản đồ trong chi tiết POI để kiểm tra hoặc ghim vị trí quán.

Mobile App: Hiển thị danh sách quán ăn theo khoảng cách (Nearby) và hỗ trợ chỉ đường từ vị trí hiện tại của thực khách.

4.7. Module Mã QR & Thuyết minh tự động (QR & TTS)

Mã QR định danh: Hệ thống tự động sinh mã QR duy nhất cho mỗi POI dựa trên Id. Chủ quán có thể tải mã này về để dán tại quán.

Quét QR & TTS: Khi người dùng dùng Mobile App quét mã QR:
Ứng dụng truy xuất dữ liệu POI và tự động kích hoạt tính năng Text-to-Speech (TTS).
Giọng đọc sẽ thuyết minh thông tin mô tả của quán dựa trên ngôn ngữ người dùng đã chọn (vi/en).

4.8. Module Hàng rào địa lý (Geofencing)

Vùng ẩm thực thông minh: Thiết lập bán kính xung quanh tọa độ POI (ví dụ: 100m-200m).
Thông báo chủ động: Khi thực khách di chuyển vào vùng Geofence, Mobile App sẽ gửi thông báo đẩy (Push Notification) mời gọi khám phá hoặc giới thiệu các món đặc sắc của quán gần đó.

5. Quy ước Trạng thái Hệ thống (State Machine)

5.1. Trạng thái của Địa điểm (POI Status)

Hệ thống vận hành POI dựa trên 4 trạng thái cốt lõi:

- pending (Chờ duyệt): Trạng thái mặc định khi Owner tạo mới hoặc khi khôi phục một POI đã xóa.
- approved (Đã duyệt): Admin đã chấp thuận, POI được phép hiển thị public trên Mobile App.
- rejected (Từ chối): Admin từ chối phê duyệt POI.
- deleted (Đã xóa): POI bị xóa mềm (Bởi Admin hoặc Owner). Bị ẩn mặc định trên mọi API danh sách/tìm kiếm xung quanh, trừ khi có cờ includeDeleted=true.

6. Lộ trình cải tiến tương lai (Future Roadmap)

Dựa trên kiến trúc hiện tại, các tính năng sau được đề xuất cho các Phase tiếp theo:

- Nâng cấp Audit Log: Thay vì chỉ dùng chuỗi status = deleted, bổ sung thêm trường boolean isDeleted và timestamp deletedAt để dễ dàng truy xuất lịch sử.

- Tách biệt UI Thùng rác (Trash): Xây dựng một trang "Thùng rác" riêng biệt trên Admin/Owner UI thay vì gộp chung với danh sách POI đang hoạt động bằng cờ includeDeleted.
- Chuẩn hóa Model User: Tách biệt trường Email độc lập khỏi trường Username trong UserModel để đảm bảo hệ thống gửi thông báo chính xác và hỗ trợ đăng nhập linh hoạt hơn.
- Đảm bảo chất lượng (QA): Bổ sung Automation Integration Tests đặc biệt cho các luồng API chuyển đổi trạng thái (Approve, Reject, Delete, Restore) để tránh lỗi logic.

7. Yêu cầu Phi chức năng (Non-Functional Requirements)

CORS: Cấu hình mở (AllowAnyOrigin) để hỗ trợ kết nối đa nền tảng (Web, Mobile, Localhost).
Dependency Injection (DI): Đảm bảo sử dụng chung một IMongoDatabase instance để tối ưu tài nguyên và tránh lỗi kết nối.
Swagger: Cung cấp tài liệu API tự động cho việc kiểm thử và tích hợp.

8. Luồng Mobile App (Mobile Application Flow)

Tương tác: Lấy danh sách POI dựa trên vị trí gần nhất hoặc theo ngôn ngữ tùy chỉnh
Tracking: Tự động gửi yêu cầu POST /api/PoiVisit khi người dùng xem chi tiết để ghi nhận mức độ quan tâm.
Hiển thị: Tự động ẩn các POI có trạng thái deleted hoặc chưa được duyệt khỏi giao diện người dùng cuối.

9. Checklist Kiểm thử (Quality Assurance Checklist)

Kiểm tra API: Đảm bảo các endpoint GET trả về đúng dữ liệu khi có/không có cờ includeDeleted.
Kiểm tra Logic: Xác nhận POI sau khi xóa mềm (deleted) vẫn có thể khôi phục lại trạng thái pending.
Kiểm tra Email: Thử nghiệm việc gửi mail khi Admin duyệt hoặc từ chối POI.

10. Quy tắc Dữ liệu & Ràng buộc Logic (Data Rules & Constraints)

Quy tắc sở hữu POI: Chỉ có tài khoản có OwnerId mới được phép tạo POI. Admin không trực tiếp tạo POI trên giao diện để đảm bảo tính khách quan và quy trình kiểm duyệt.

Ràng buộc trạng thái:

- POI mới tạo luôn mặc định là pending.
- Hành động khôi phục (Restore) một POI đã xóa sẽ đưa POI đó quay lại trạng thái pending để chờ duyệt lại, không đưa thẳng lên approved.

Tính toàn vẹn Email: Nếu Username của người dùng không phải định dạng email, hệ thống sẽ tự động bỏ qua (skip) việc gửi mail thay vì gây lỗi hệ thống (exception).

11. Thiết kế Luồng Dữ liệu (API Data Flow)

Luồng Truy vấn (Read):

- Endpoint mặc định (GET /api/Poi) sẽ tự động lọc bỏ các POI có trạng thái deleted để bảo vệ dữ liệu người dùng cuối.
- Tham số includeDeleted=true là bắt buộc nếu muốn xem dữ liệu trong các trang quản lý của Admin hoặc Owner.
- Luồng Ghi (Write): Mọi hành động cập nhật thông tin POI qua PUT sẽ tự động ghi đè dữ liệu vào phân vùng ngôn ngữ Tiếng Việt (Localizations["vi"]) và cập nhật đồng thời các trường phẳng như ảnh và vị trí.

12. Kế hoạch Triển khai & Cấu hình (Deployment & Configuration)

Quản trị biến môi trường (SMTP): Cần cấu hình chính xác các khóa trong appsettings.json để hệ thống thông báo hoạt động:

- Email:SmtpHost, Email:SmtpPort: Máy chủ gửi mail.
- Email:FromEmail, Email:Password: Tài khoản gửi.
- Email:NotificationEmail: Email nhận thông báo hệ thống (tùy chọn).

Cơ sở dữ liệu: Đảm bảo IMongoDatabase được đăng ký dạng Singleton trong Program.cs để duy trì kết nối ổn định xuyên suốt vòng đời ứng dụng.

13. Ma trận Phê duyệt & Email (Approval & Email Matrix)

Hành động	Trạng thái POI	Đối tượng nhận mail	Nội dung Mail
Tạo mới	pending	Owner & Hệ thống	Thông báo có POI mới chờ duyệt
Duyệt (Approve)	approved	Owner	Thông báo POI đã được công khai
Từ chối (Reject)	rejected	Owner	Thông báo lý do từ chối (Rejection template)
Xóa (Delete)	deleted	Owner	Thông báo POI đã bị gỡ bỏ
Khóa User	IsActive = false	Người dùng bị khóa	Thông báo trạng thái tài khoản

14. Quản lý Lỗi & Phản hồi Hệ thống (Error Handling & System Feedback)

API Responses: Hệ thống phải trả về mã lỗi HTTP chuẩn (400 cho lỗi dữ liệu, 401/403 cho lỗi quyền, 404 cho không tìm thấy, 500 cho lỗi server).
User Feedback (Frontend): * Sử dụng MudBlazor Snackbar để hiển thị thông báo thành công/thất bại sau mỗi thao tác (VD: "Đã gửi yêu cầu duyệt POI thành công").
Hiển thị Loading Overlay khi đang xử lý các tác vụ nặng như lọc dữ liệu thống kê hoặc upload ảnh.

Validation: Kiểm tra dữ liệu đầu vào (Input Validation) cả ở phía Client (Blazor/MAUI) và Server (FluentValidation) để ngăn chặn dữ liệu rác.

15. Chỉ số hiệu suất chính (Key Performance Indicators - KPIs)

Để đo lường sự thành công của dự án, hệ thống cần theo dõi:
Thời gian phản hồi API: Mục tiêu dưới 500ms cho các truy vấn danh sách POI thông thường.
Tỷ lệ tăng trưởng POI: Số lượng POI mới được tạo hàng tháng.
Tỷ lệ giữ chân người dùng: Theo dõi qua số lượt truy cập (Visits) định kỳ trên từng POI.
Hiệu quả kiểm duyệt: Thời gian trung bình từ khi POI ở trạng thái pending đến khi được approved hoặc rejected.

16. Nguyên tắc Thiết kế Giao diện (UX/UI Design Principles)

Tính nhất quán: Sử dụng hệ màu và icon chuẩn của MudBlazor trên toàn bộ trang Admin và Owner.
Mobile-First (cho App): Giao diện Mobile App phải tối ưu cho thao tác một tay, các nút bấm chi tiết POI phải rõ ràng.
Bố cục Dashboard:

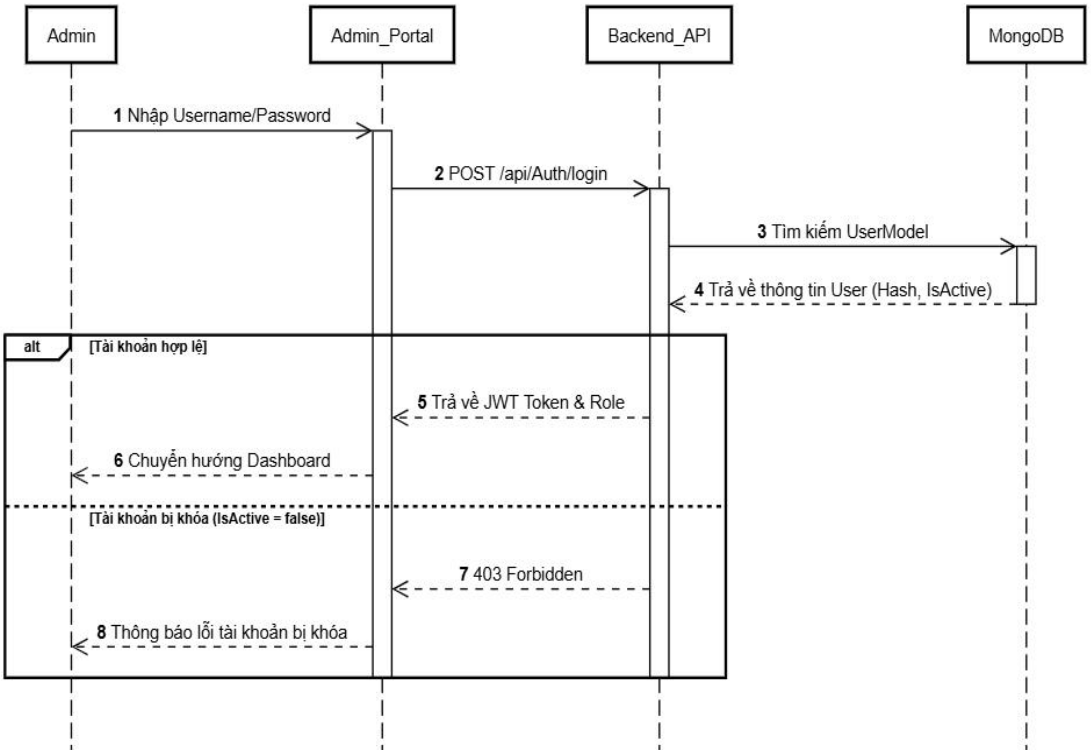
- Thông tin quan trọng nhất (KPIs) nằm ở trên cùng.
- Các biểu đồ thống kê trực quan (biểu đồ tròn cho danh mục, biểu đồ cột cho lượt visit).

Khả năng tiếp cận: Đảm bảo độ tương phản màu sắc và kích thước chữ dễ đọc cho mọi đối tượng thực khách.

17. Sequence Diagram

17.1. Khởi động và Kiểm tra kết nối (Startu1. Đăng nhập và Kiểm tra quyền (Authentication)

Xác thực danh tính và kiểm tra trạng thái hoạt động của tài khoản.



Luồng hoạt động chi tiết (Step-by-Step)

- Nhập thông tin đăng nhập: Người dùng (Admin/Owner) nhập Username và Password vào giao diện Admin Portal. Đây là bước khởi đầu để thiết lập ngữ cảnh người dùng.
- Gửi yêu cầu xác thực: Admin Portal gửi một yêu cầu POST /api/Auth/login kèm theo thông tin đăng nhập đến Backend API. Thanh tiến trình (activation bar) bắt đầu hoạt động tại API để xử lý yêu cầu này.
- Truy vấn thông tin người dùng: Backend API thực hiện truy vấn vào MongoDB để tìm kiếm bản ghi UserModel khớp với Username.
- Phản hồi dữ liệu gốc: MongoDB trả về thông tin người dùng bao gồm chuỗi mật khẩu đã mã hóa (PasswordHash) và trạng thái tài khoản (IsActive).
- Kiểm tra logic tại Backend: Tại đây hệ thống rẽ nhánh (khối alt) để xử lý hai kịch bản:

Kịch bản 1 - Tài khoản hợp lệ: Nếu mật khẩu khớp và IsActive = true, Backend tạo mã JWT Token chứa quyền hạn (Role) và gửi về cho Portal.

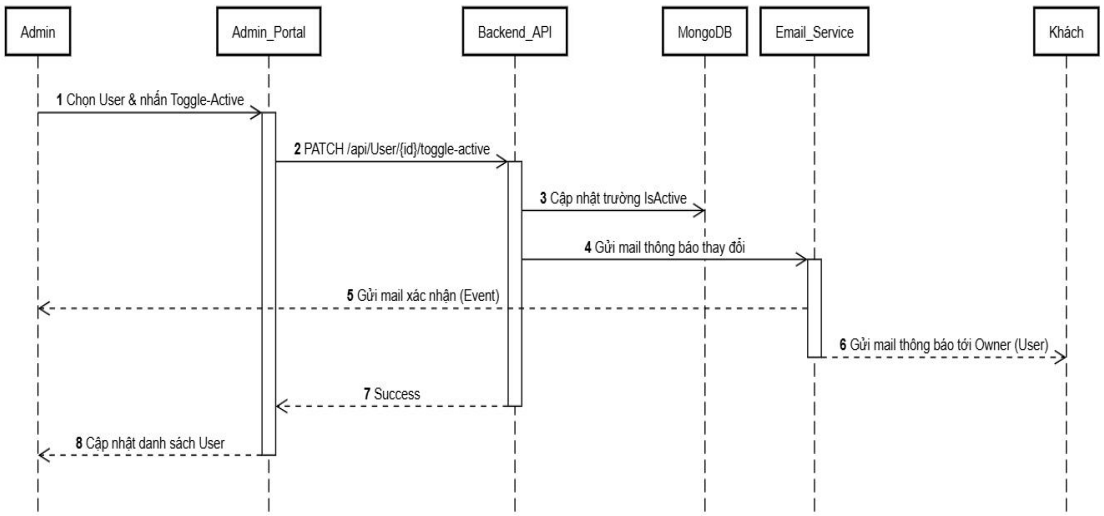
Kịch bản 2 - Tài khoản bị khóa: Nếu IsActive = false, Backend lập tức ngắt quá trình và trả về mã lỗi 403 Forbidden.

- Kết thúc và Phản hồi giao diện:

Nếu thành công: Portal lưu Token và Chuyển hướng người dùng vào trang Dashboard.

Nếu thất bại: Portal nhận mã 403 và Hiển thị thông báo lỗi "Tài khoản bị khóa" để người dùng nhận biết.

17.2. Tải danh sách địa điểm - Thành công (Load Data Success)



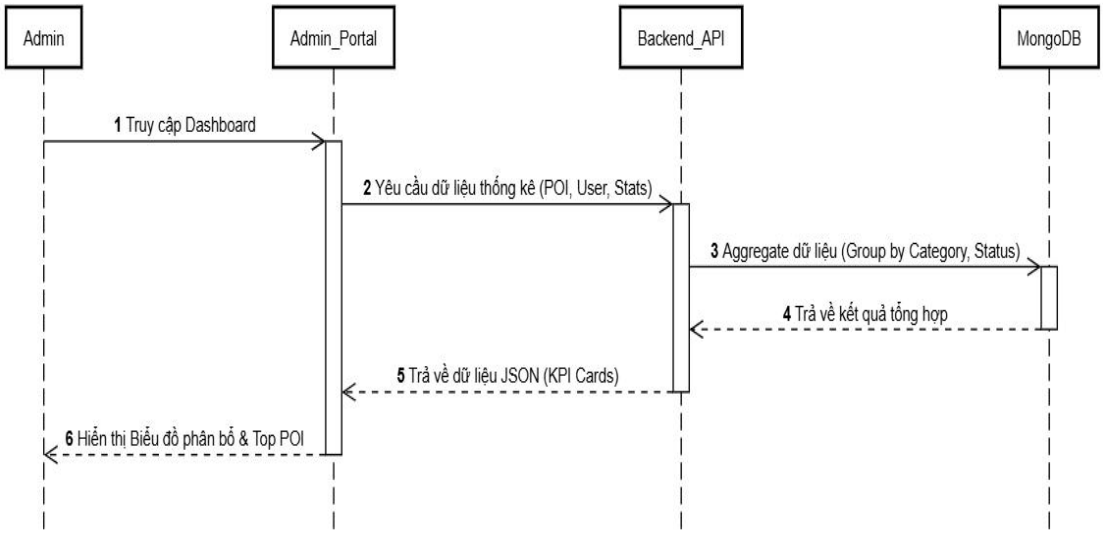
Luồng hoạt động chi tiết (Step-by-Step)

- Chọn User & Kích hoạt: Tại giao diện quản lý người dùng (UserManager.razor), Admin tìm đối tượng cần xử lý và nhấn nút chuyển đổi trạng thái (Toggle-Active).
- Gửi yêu cầu thay đổi: Admin_Portal gửi một yêu cầu PATCH /api/User/{id}/toggle-active tới Backend_API.
- Cập nhật Cơ sở dữ liệu: Backend_API tiếp nhận yêu cầu và thực hiện cập nhật trường IsActive (từ true sang false hoặc ngược lại) trong bộ lưu trữ MongoDB.
- Kích hoạt luồng Email: Sau khi dữ liệu được cập nhật thành công, Backend_API gọi đến Email_Service thông qua phương thức SendUserAccountEventEmailAsync để thông báo về sự thay đổi này.
- Gửi mail xác nhận (Event): Email_Service gửi một email thông báo sự kiện (event) tới Admin (hoặc hệ thống giám sát) để ghi nhận hành động quản trị.

- Gửi mail thông báo tới Owner: Đồng thời, Email_Service gửi một email trực tiếp tới người dùng (Owner) để thông báo trạng thái tài khoản của họ đã thay đổi.
- Phản hồi thành công: Backend_API gửi mã phản hồi thành công (Success) về cho Admin_Portal.
- Cập nhật giao diện: Admin_Portal nhận phản hồi và làm mới (Refresh) danh sách người dùng trên màn hình để hiển thị trạng thái mới nhất.

17.3. Dashboard Tổng quát (Admin)

Hệ thống tổng hợp dữ liệu toàn diện về POI và người dùng.



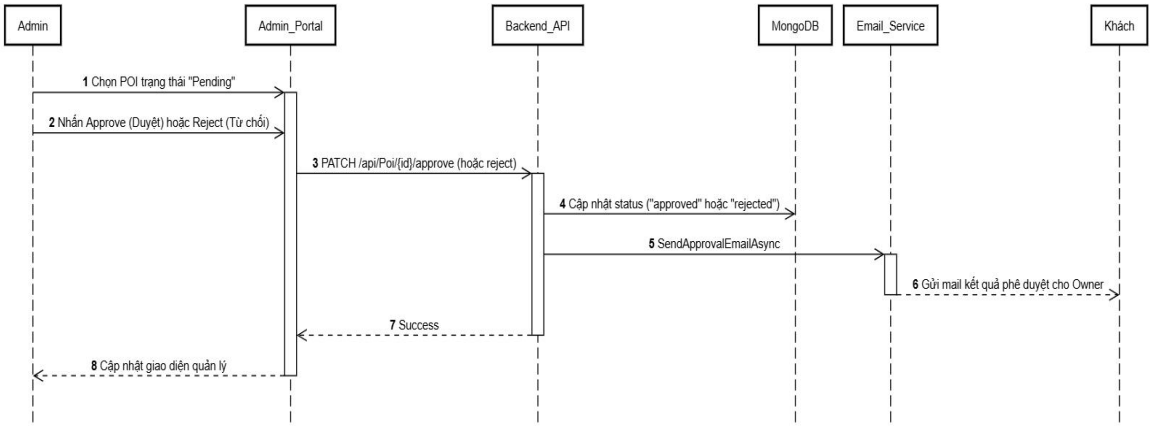
Luồng hoạt động chi tiết (Step-by-Step)

- Truy cập Dashboard: Admin thực hiện đăng nhập và truy cập vào trang quản trị tổng thể. Đây là bước kích hoạt yêu cầu lấy dữ liệu để hiển thị trạng thái hệ thống.
- Yêu cầu dữ liệu thống kê: Giao diện Admin_Portal (Blazor) tự động gửi một yêu cầu lấy thông tin tổng hợp (bao gồm số lượng POI, Users và các chỉ số truy cập) tới Backend_API.
- Aggregate dữ liệu: Backend nhận yêu cầu và thực hiện các câu lệnh truy vấn tập hợp (Aggregation) phức tạp trên MongoDB. Tại đây, dữ liệu được gom nhóm theo danh mục (Category) và trạng thái (Status).

- Trả về kết quả tổng hợp: MongoDB xử lý xong và trả về các tập hợp dữ liệu đã được tính toán (ví dụ: tổng số quán ăn theo từng loại, số lượng tài khoản đang bị khóa) cho Backend.
- Trả về dữ liệu JSON: Backend_API đóng gói các kết quả này thành định dạng JSON chuyên dụng cho các thẻ chỉ số (KPI Cards) và gửi về cho phía giao diện.
- Hiển thị Biểu đồ & Top POI: Admin_Portal nhận dữ liệu và vẽ thành các biểu đồ trực quan, hiển thị các thẻ thông báo (KPI) và danh sách các địa điểm được quan tâm nhất (Top POI) dựa trên số lượt truy cập thực tế.

17.4. Phê duyệt hoặc Từ chối POI (Admin)

Quy trình kiểm duyệt nội dung do chủ quán đăng tải.



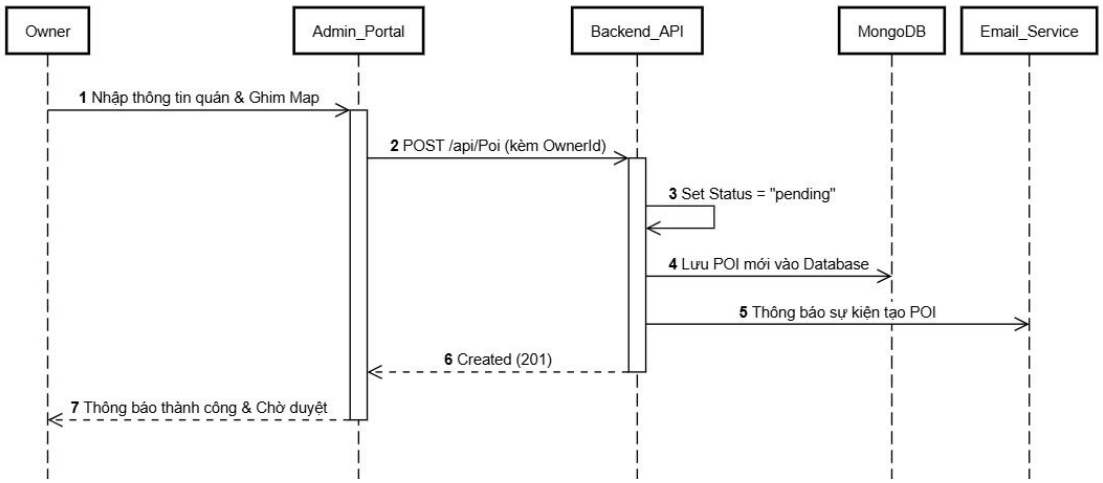
Luồng hoạt động chi tiết (Step-by-Step)

- Chọn POI chờ duyệt: Admin truy cập vào trang quản lý địa điểm (PoiManager.razor) và chọn một địa điểm đang ở trạng thái "Pending" (Chờ duyệt).
- Ra quyết định: Admin xem xét nội dung và nhấn nút Approve (Duyệt) hoặc Reject (Từ chối) trên giao diện.
- Gửi yêu cầu cập nhật: Admin_Portal gửi một yêu cầu PATCH /api/Poi/{id}/approve (hoặc /reject) tới Backend_API.
- Cập nhật trạng thái trong Database: Backend_API thực hiện thay đổi trường status của POI trong MongoDB sang giá trị tương ứng là "approved" hoặc "rejected".

- Kích hoạt dịch vụ Email: Sau khi lưu thành công, Backend gọi phương thức SendApprovalEmailAsync của Email_Service.
- Thông báo cho Chủ sở hữu (Owner): Email_Service thực hiện gửi một email kết quả (chấp nhận hoặc từ chối) tới địa chỉ email của Chủ quán (Khách/Owner) để họ nắm bắt tình trạng.
- Phản hồi thành công: Backend_API gửi mã phản hồi thành công (Success) về cho giao diện quản trị.
- Cập nhật giao diện: Admin_Portal làm mới dữ liệu và hiển thị trạng thái mới của địa điểm trên danh sách quản lý.

17.5. Tạo Địa điểm ẩm thực mới (Owner)

Mỗi địa điểm mới tạo luôn mặc định ở trạng thái chờ duyệt.



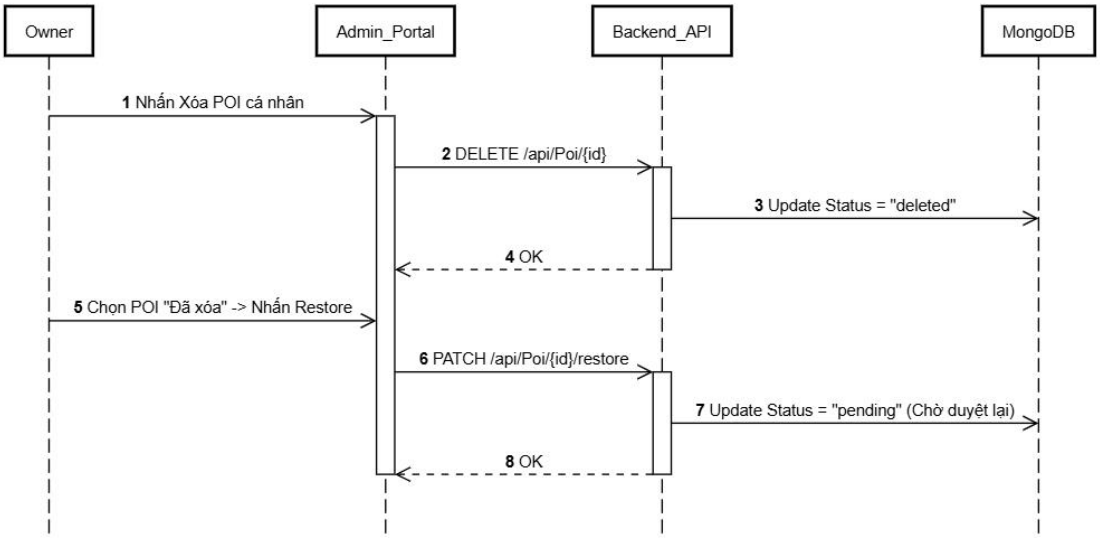
Luồng hoạt động chi tiết (Step-by-Step)

- Nhập thông tin & Ghim Map: Owner truy cập vào giao diện quản lý cá nhân (MyPois.razor), điền đầy đủ thông tin quán ăn, tải ảnh và chọn vị trí chính xác trên bản đồ.
- Gửi yêu cầu tạo mới: Admin_Portal gửi một yêu cầu POST /api/Poi kèm theo thông tin địa điểm và mã định danh chủ sở hữu (OwnerId) tới Backend_API.
- Thiết lập trạng thái chờ: Tại server, Backend_API tự động thiết lập thuộc tính Status = "pending" cho địa điểm này. Điều này đảm bảo quán mới không xuất hiện ngay lập tức trên app của khách khi chưa được Admin phê duyệt.

- Lưu vào Cơ sở dữ liệu: Hệ thống thực hiện ghi bản ghi POI mới vào bộ lưu trữ MongoDB.
- Thông báo sự kiện: Sau khi lưu thành công, Backend_API gọi đến Email_Service để gửi thông báo về sự kiện tạo POI mới tới hệ thống quản trị và gửi mail cho Owner (nếu username là email hợp lệ).
- Xác nhận khởi tạo: Backend gửi phản hồi mã 201 Created về cho phía giao diện, xác nhận dữ liệu đã được tiếp nhận thành công.
- Thông báo hoàn tất: Admin_Portal hiển thị thông báo cho Owner biết quán ăn đã được gửi đi thành công và đang trong trạng thái "Chờ duyệt".

17.6. Xóa mềm và Khôi phục POI (Owner/Admin)

Dữ liệu không bị xóa vĩnh viễn mà chuyển trạng thái để quản lý.



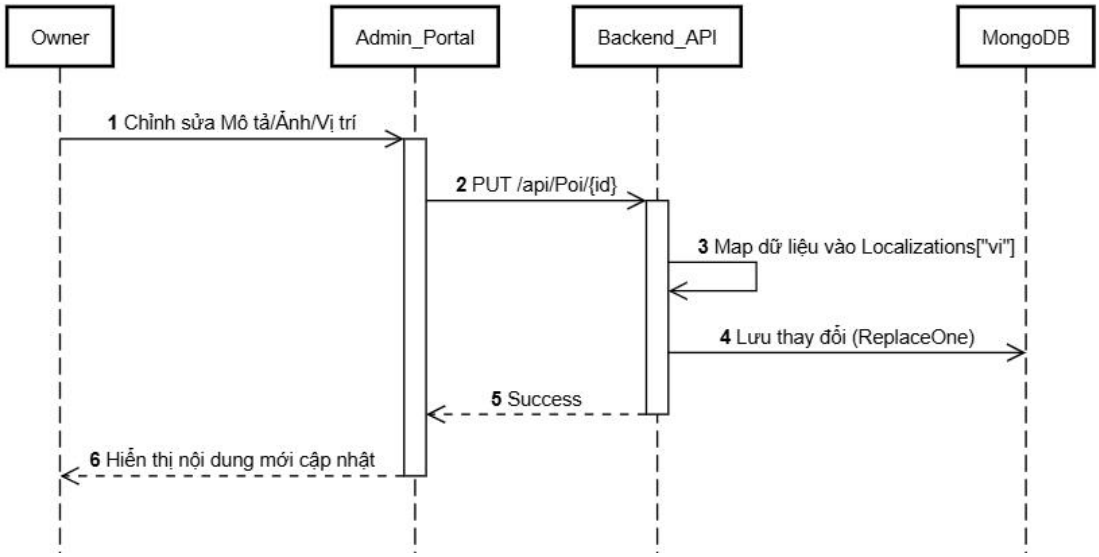
Luồng hoạt động chi tiết (Step-by-Step)

- Yêu cầu xóa POI: Owner truy cập danh sách địa điểm cá nhân và nhấn nút xóa một POI cụ thể.
- Gửi lệnh xóa: Admin_Portal gửi một yêu cầu DELETE /api/Poi/{id} tới Backend_API.
- Xử lý xóa mềm: Tại server, hệ thống không thực hiện xóa vĩnh viễn dữ liệu. Thay vào đó, Backend_API gửi lệnh tới MongoDB để cập nhật trường status = "deleted" cho POI đó.

- Xác nhận xóa: Sau khi cập nhật DB thành công, Backend gửi mã phản hồi OK về cho Portal để cập nhật giao diện người dùng.
- Yêu cầu khôi phục: Để lấy lại quán đã xóa, Owner vào mục danh sách đã xóa (thường dùng tham số includeDeleted=true) và nhấn nút Restore.
- Gửi lệnh khôi phục: Admin_Portal gửi một yêu cầu PATCH /api/Poi/{id}/restore tới Backend_API.
- Cập nhật trạng thái chờ duyệt: Backend_API cập nhật lại trạng thái POI trong MongoDB thành status = "pending". Điều này có nghĩa là địa điểm sau khi khôi phục phải chờ Admin duyệt lại từ đầu mới được hiển thị công khai.
- Xác nhận khôi phục: Backend gửi mã phản hồi OK về cho Portal để hiển thị thông báo khôi phục thành công cho người dùng.

17.7. Cập nhật thông tin & Đồng bộ Ngôn ngữ (Owner)

Dữ liệu phẳng được đồng bộ vào phân vùng ngôn ngữ tương ứng.



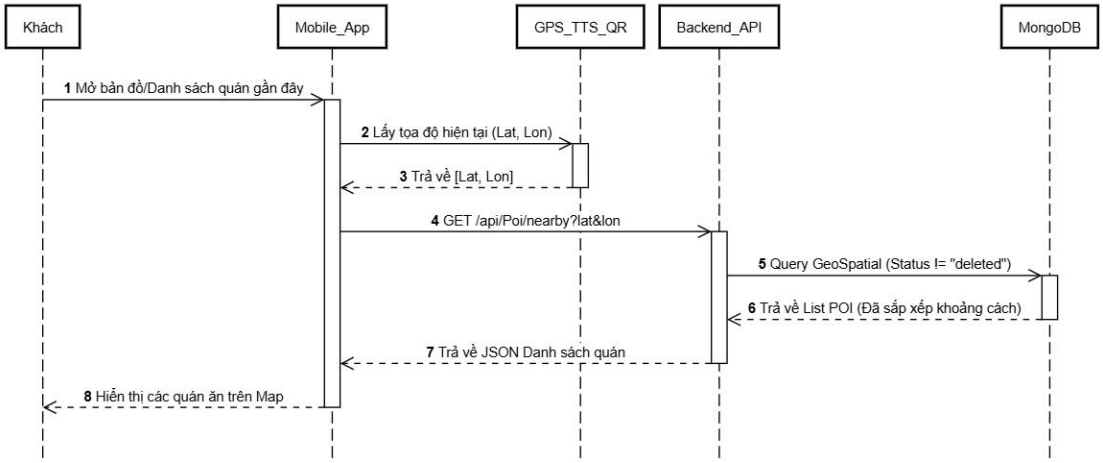
Luồng hoạt động chi tiết (Step-by-Step)

- Chỉnh sửa nội dung: Owner truy cập vào mục quản lý địa điểm cá nhân (MyPois.razor) và thực hiện thay đổi các thông tin như Mô tả, Ảnh hoặc Vị trí của quán trên bản đồ.

- Gửi yêu cầu cập nhật: Giao diện Admin_Portal (Blazor) gửi một yêu cầu PUT /api/Poi/{id} chứa dữ liệu mới tới Backend_API.
- Ánh xạ ngôn ngữ (Mapping): Tại server, Backend_API tiếp nhận dữ liệu phẳng từ client và thực hiện ánh xạ (map) các thông tin văn bản vào trường dữ liệu đa ngôn ngữ Localizations["vi"]. Điều này đảm bảo nội dung tiếng Việt của quán được cập nhật chính xác trong cấu trúc dữ liệu.
- Lưu thay đổi: Backend thực hiện lệnh ReplaceOne (thay thế toàn bộ bản ghi cũ bằng bản ghi mới đã cập nhật) vào bộ lưu trữ MongoDB.
- Xác nhận thành công: Sau khi Database phản hồi thành công, Backend_API gửi mã trạng thái Success về cho phía giao diện.
- Cập nhật hiển thị: Admin_Portal làm mới màn hình và hiển thị các nội dung vừa được cập nhật cho Owner xem lại.

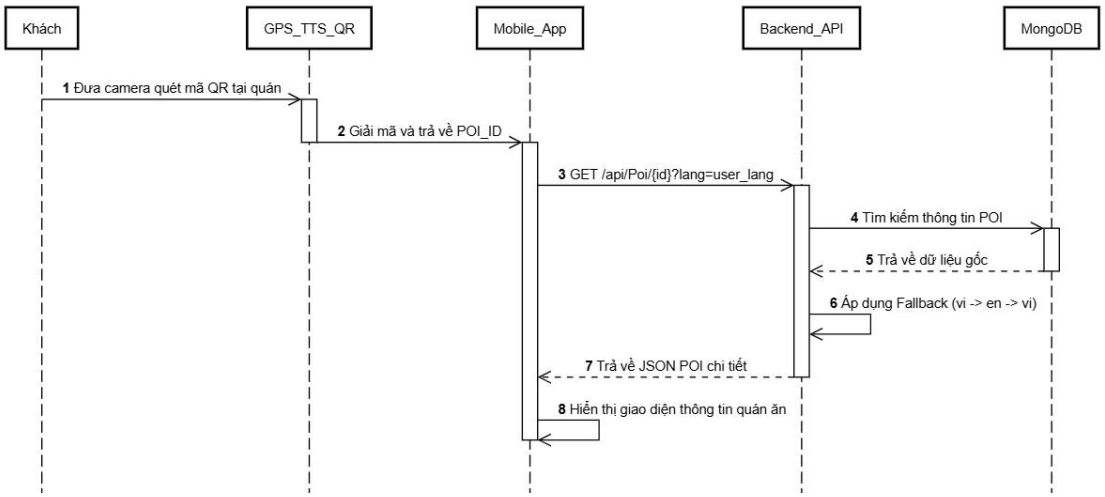
17.8.Tìm kiếm quán ăn xung quanh (Mobile User)

Hệ thống lọc theo vị trí và bỏ qua các quán đã bị xóa.



17.9. Luồng Quét mã QR (QR Code Scanning Flow)

Sơ đồ này tập trung vào việc định danh POI và truy xuất dữ liệu từ hệ thống ngay sau khi quét mã vật lý.



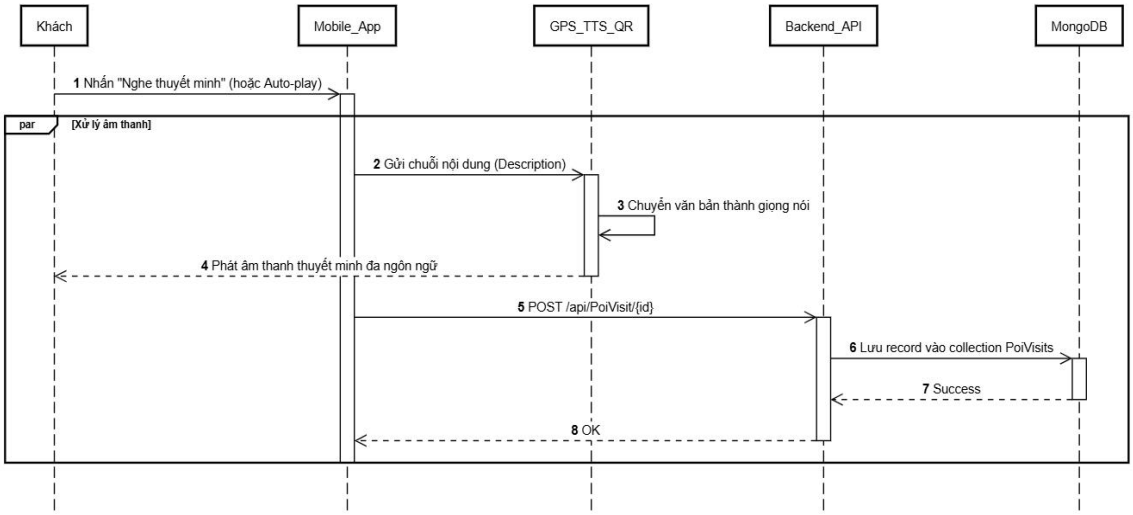
Luồng hoạt động chi tiết (Step-by-Step)

- Quét mã QR: Khách hàng đưa camera điện thoại để quét mã QR vật lý đặt tại quán ăn thông qua module phần cứng/phần mềm xử lý (GPS_TTS_QR).
- Giải mã ID: Hệ thống GPS_TTS_QR thực hiện giải mã hình ảnh thành chuỗi dữ liệu và trả về mã định danh địa điểm (POI_ID) cho ứng dụng di động (Mobile_App).
- Gửi yêu cầu lấy dữ liệu: Mobile_App gửi một yêu cầu GET /api/Poi/{id} kèm theo ngôn ngữ hiện tại của người dùng (user_lang) tới Backend_API.
- Truy vấn dữ liệu: Backend_API tiếp nhận yêu cầu và thực hiện tìm kiếm thông tin chi tiết của địa điểm đó trong cơ sở dữ liệu MongoDB.
- Phản hồi dữ liệu gốc: MongoDB trả về đối tượng dữ liệu thô (chứa đầy đủ các bản dịch và thông tin vị trí) cho Backend.
- Áp dụng Fallback ngôn ngữ: Backend thực hiện hàm HydrateAndFallback. Nếu ngôn ngữ yêu cầu không có sẵn, hệ thống tự động ưu tiên lấy dữ liệu theo thứ tự: Tiếng Việt (vi) -> Tiếng Anh (en) -> Tiếng Việt (vi) để đảm bảo không bị trống nội dung.
- Trả về dữ liệu chi tiết: Sau khi xử lý xong ngôn ngữ, Backend_API gửi phản hồi chứa dữ liệu JSON đã được làm sạch và chuẩn hóa về cho Mobile_App.

- Hiển thị giao diện: Mobile_App nhận dữ liệu và trình bày các thông tin như tên quán, mô tả, hình ảnh lên màn hình điện thoại cho khách hàng xem.

17.10. Luồng Thuyết minh tự động (TTS Narration Flow)

Sơ đồ này mô tả cách ứng dụng chuyển đổi văn bản mô tả thành giọng nói và đồng thời ghi nhận lượt tương tác của người dùng.



Luồng hoạt động chi tiết (Step-by-Step)

- Kích hoạt thuyết minh: Sau khi xem thông tin quán ăn, Khách nhấn nút "Nghe thuyết minh" hoặc ứng dụng tự động kích hoạt chế độ Auto-play. Đây là lệnh bắt đầu cho chuỗi xử lý đa phương tiện.
- Xử lý song song (Parallel Processing): Hệ thống mở ra một khối par, cho phép thực hiện đồng thời hai luồng công việc độc lập để tiết kiệm thời gian và tối ưu hiệu suất:

Luồng 1: Chuyển đổi âm thanh (TTS): * Mobile_App gửi chuỗi nội dung văn bản (Description) tới module xử lý GPS_TTS_QR.

- Module này thực hiện "Chuyển văn bản thành giọng nói" ngay tại chỗ.
- Âm thanh thuyết minh đa ngôn ngữ được phát ra cho Khách nghe.

Luồng 2: Ghi nhận thống kê (Analytics):

- Mobile_App gửi một yêu cầu POST /api/PoiVisit/{id} tới Backend_API.

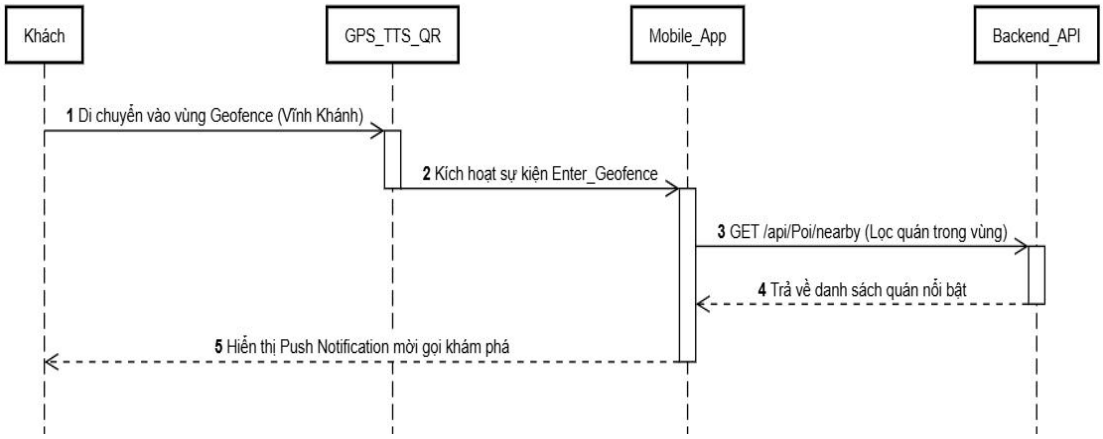
- Backend_API thực hiện lưu một bản ghi vào bộ sưu tập PoiVisits trong MongoDB. Việc này giúp hệ thống biết được quán nào đang "hot" để đưa lên Dashboard.

Phản hồi xác nhận: Sau khi ghi dữ liệu xong, MongoDB trả về kết quả thành công cho Backend, và Backend gửi mã OK về cho Mobile_App.

Hoàn tất trải nghiệm: Người dùng vừa được nghe thuyết minh một cách mượt mà, trong khi hệ thống đã âm thầm hoàn thành việc thu thập dữ liệu thống kê mà không gây ra bất kỳ sự chậm trễ nào cho giao diện.

17.11. Hàng rào địa lý - Geofence (Mobile User)

Tự động nhận diện khi thực khách đi vào vùng ẩm thực đặc biệt.



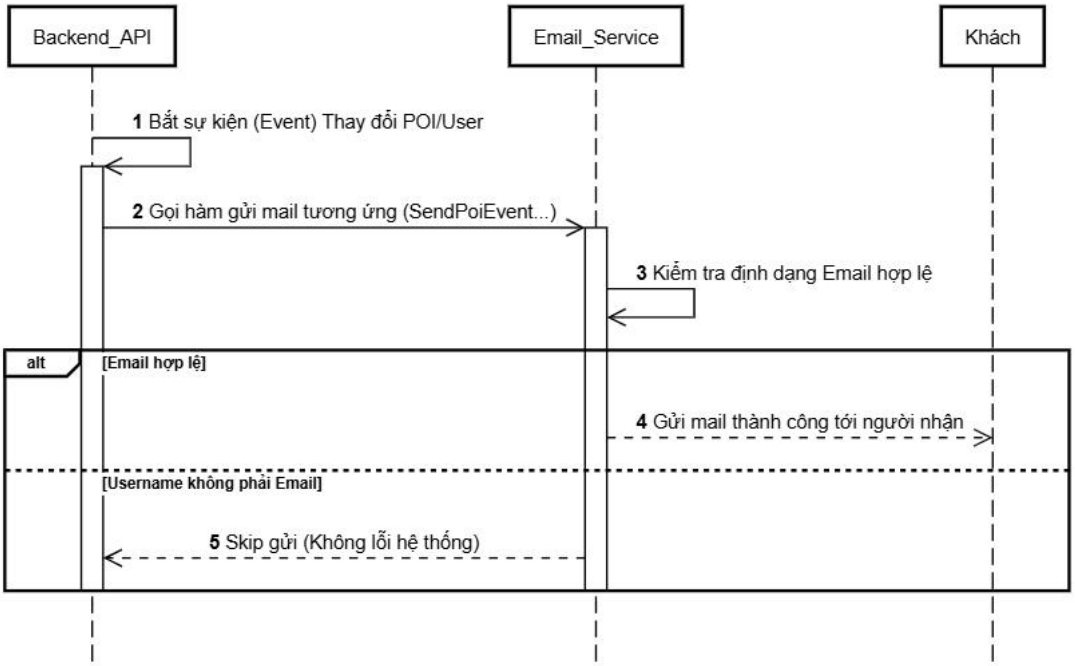
Luồng hoạt động chi tiết (Step-by-Step)

- Di chuyển vào vùng Geofence: Khách hàng (mang theo điện thoại có cài ứng dụng) di chuyển vào khu vực địa lý đã được thiết lập "hàng rào" ảo (ví dụ: Phố ẩm thực Vĩnh Khánh).
- Kích hoạt sự kiện (Trigger): Module phần cứng/hệ điều hành GPS_TTS_QR nhận diện tọa độ GPS và kích hoạt sự kiện Enter_Geofence, thông báo cho Mobile_App rằng người dùng đã vào vùng mục tiêu.
- Yêu cầu lọc quán ăn: Mobile_App ngay lập tức gửi một yêu cầu GET /api/Poi/nearby tới Backend_API. Yêu cầu này kèm theo tọa độ hiện tại để hệ thống lọc ra các quán ăn nằm trong vùng Geofence đó.

- Trả về danh sách: Backend_API truy vấn cơ sở dữ liệu và trả về danh sách các địa điểm nổi bật hoặc đang có chương trình khuyến mãi trong khu vực khách đang đứng.
- Gửi thông báo đẩy (Push Notification): Mobile_App nhận danh sách và hiển thị một thông báo trên màn hình điện thoại (ví dụ: "Chào mừng bạn đến với Vĩnh Khánh! Khám phá ngay các quán ốc nổi tiếng xung quanh đây").

17.12. Gửi Email thông báo sự kiện (Backend)

Luồng xử lý ngầm mỗi khi có biến động về dữ liệu.



Luồng hoạt động chi tiết (Step-by-Step)

- Bắt sự kiện (Event): Khi có bất kỳ thay đổi nào xảy ra trong hệ thống (như tạo mới POI, phê duyệt địa điểm, hoặc khóa tài khoản người dùng), Backend_API sẽ tự động nhận diện và kích hoạt một sự kiện thông báo.
- Gọi dịch vụ Email: Backend gọi hàm gửi mail tương ứng từ Email_Service (ví dụ: SendPoiEventEmailAsync hoặc SendUserAccountEventEmailAsync).

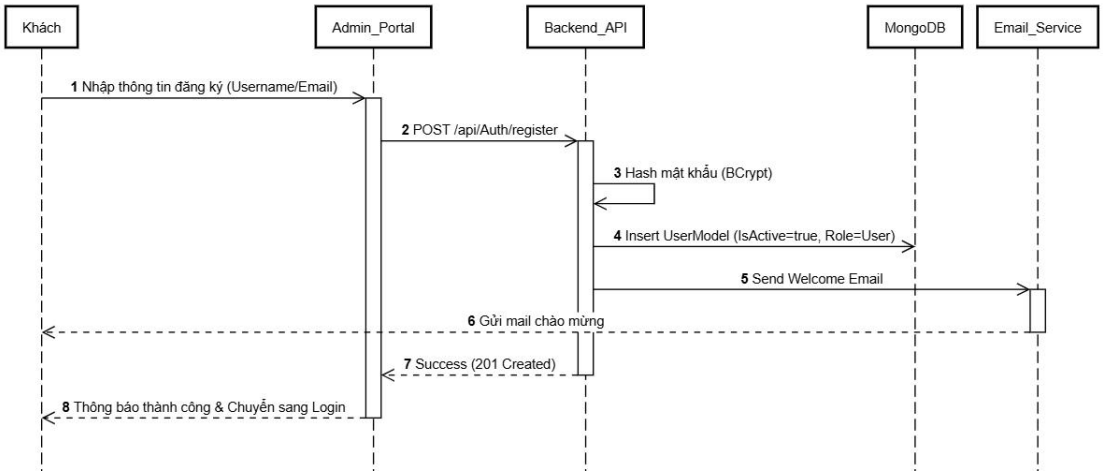
- Kiểm tra định dạng: Trước khi gửi, Email_Service thực hiện kiểm tra xem Username (thường là định danh người dùng) có phải là một địa chỉ Email hợp lệ hay không.
- Xử lý logic (Khối alt):

✧ Trường hợp Email hợp lệ: Dịch vụ thực hiện gửi nội dung thông báo thành công tới hòm thư của Khách/Người nhận. Đây là lúc người dùng nhận được thông tin về trạng thái POI hoặc tài khoản của họ.

✧ Trường hợp không phải Email: Nếu hệ thống phát hiện Username không phải định dạng email (ví dụ: chỉ là một chuỗi ký tự ID), nó sẽ thực hiện lệnh Skip (Bỏ qua). Hành động này giúp hệ thống không bị treo hoặc báo lỗi không cần thiết khi gặp dữ liệu không chuẩn.

17.13. Luồng Đăng ký tài khoản (Registration)

Mô tả cách một người dùng mới (tiềm năng là Owner) gia nhập hệ thống.



Luồng hoạt động chi tiết (Step-by-Step)

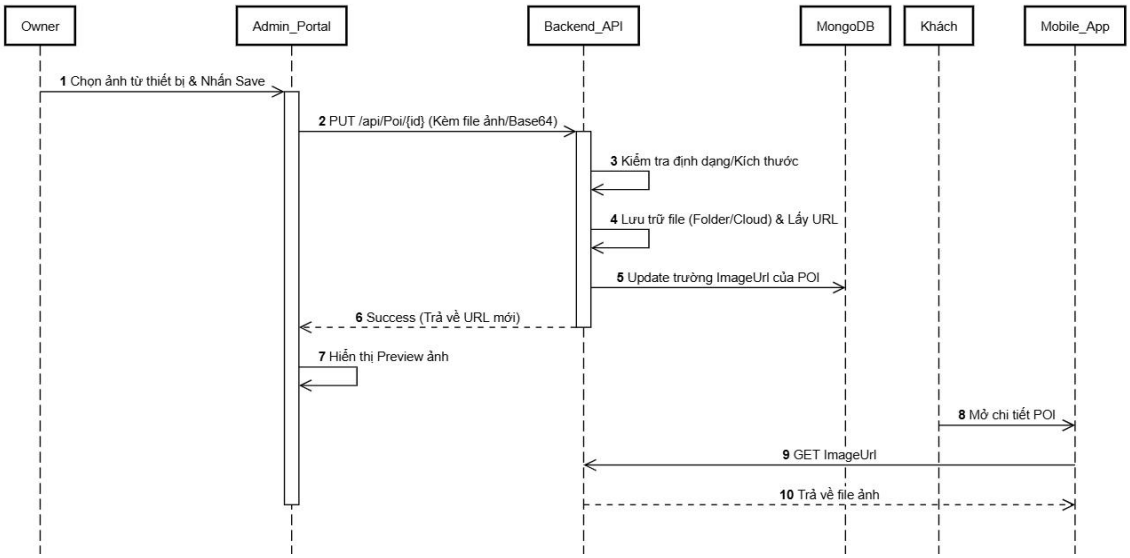
- Nhập thông tin đăng ký: Người dùng (Khách) truy cập vào giao diện đăng ký trên Admin_Portal, nhập các thông tin cần thiết như Username/Email và mật khẩu.
- Gửi yêu cầu đăng ký: Portal gửi một yêu cầu POST /api/Auth/register chứa thông tin đăng ký tới Backend_API.
- Bảo mật mật khẩu (Hashing): Tại server, Backend_API sử dụng thư viện BCrypt để băm (hash) mật khẩu. Việc này đảm bảo mật khẩu

thực không bao giờ được lưu dưới dạng văn bản thuần túy, giúp bảo vệ tài khoản người dùng ngay cả khi cơ sở dữ liệu bị lộ.

- Khởi tạo tài khoản trong Database: Backend thực hiện lưu một UserModel mới vào MongoDB với các giá trị mặc định: IsActive = true (tài khoản hoạt động ngay) và Role = User (quyền hạn cơ bản).
- Gửi Email chào mừng: Sau khi lưu database thành công, Backend gọi đến Email_Service để kích hoạt luồng gửi thư chào mừng.
- Thông báo tới người dùng: Email_Service gửi một email chào mừng trực tiếp tới địa chỉ email mà người dùng đã đăng ký.
- Xác nhận khởi tạo thành công: Backend gửi phản hồi mã 201 Created về cho phía giao diện, xác nhận tài khoản đã được tạo thành công.
- Chuyển hướng đăng nhập: Admin_Portal hiển thị thông báo thành công cho người dùng và tự động chuyển hướng họ sang trang Login để bắt đầu truy cập hệ thống.

17.14. Luồng Tải lên & Hiển thị hình ảnh (Image Upload)

Xử lý khi Owner cập nhật ảnh quán ăn để hiển thị trên Mobile App.



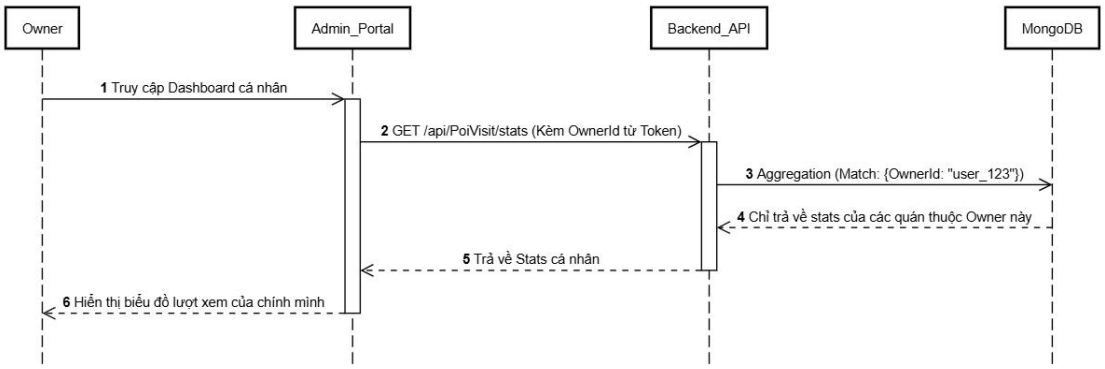
Luồng hoạt động chi tiết (Step-by-Step)

- Chọn ảnh & Lưu: Owner chọn hình ảnh quán ăn từ thiết bị cá nhân trên giao diện quản trị và nhấn nút Save.

- Gửi yêu cầu tải lên: Admin_Portal gửi một yêu cầu PUT /api/Poi/{id} chứa file ảnh dưới dạng nhị phân hoặc chuỗi Base64 tới Backend_API.
- Kiểm tra tính hợp lệ: Tại server, Backend_API thực hiện kiểm tra định dạng file (ví dụ: .jpg, .png) và kích thước file để đảm bảo không vượt quá giới hạn hệ thống.
- Lưu trữ file: Sau khi kiểm tra, Backend thực hiện lưu file vào bộ nhớ (Thư mục server hoặc Cloud Storage) và lấy về một đường dẫn truy cập duy nhất (URL).
- Cập nhật dữ liệu POI: Backend cập nhật trường ImageUrl của địa điểm tương ứng trong MongoDB bằng URL vừa lấy được.
- Xác nhận & Trả về URL: Backend gửi mã phản hồi Success kèm theo URL mới về cho phía Portal.
- Hiển thị Preview: Admin_Portal nhận URL và hiển thị ngay bản xem trước (Preview) ảnh vừa tải lên để Owner xác nhận.
- Khách xem chi tiết: Khi một người dùng (Khách) mở xem chi tiết địa điểm đó trên Mobile_App.
- Yêu cầu lấy ảnh: Mobile_App gửi yêu cầu GET tới URL hình ảnh thông qua Backend_API.
- Trả về file ảnh: Backend truy xuất file từ bộ nhớ và trả về dữ liệu hình ảnh để hiển thị lên màn hình điện thoại của khách.

17.15. Luồng Phân quyền Dashboard (Data Isolation)

Làm rõ cơ chế lọc dữ liệu (Filter) để bảo mật thông tin giữa các Owner.



Luồng hoạt động chi tiết (Step-by-Step)

- Truy cập Dashboard cá nhân: Người dùng với vai trò là Owner thực hiện đăng nhập và truy cập vào trang Dashboard dành riêng cho chủ sở hữu địa điểm.
- Yêu cầu dữ liệu kèm định danh: Admin_Portal gửi một yêu cầu GET /api/PoiVisit/stats. Điểm mấu chốt ở đây là hệ thống tự động trích xuất OwnerId từ JWT Token (mã định danh đã được cấp khi đăng nhập) để đính kèm vào yêu cầu.
- Lọc dữ liệu tại Server (Aggregation): Backend_API tiếp nhận yêu cầu và thực hiện lệnh Aggregation trên MongoDB. Tại bước này, hệ thống áp dụng bộ lọc $match theo đúng OwnerId của người đang yêu cầu.

Cơ chế phân quyền: Sơ đồ cho thấy hệ thống chỉ quét và tính toán dựa trên dữ liệu thuộc sở hữu của user_123. Điều này ngăn chặn việc Owner này xem được số liệu của Owner khác.

- Trả về kết quả riêng biệt: MongoDB chỉ trả về các số liệu thống kê (Stats) liên quan đến các quán ăn thuộc về Owner đó cho Backend.
- Gửi dữ liệu thống kê: Backend_API phản hồi kết quả thống kê cá nhân về cho phía giao diện.
- Hiển thị biểu đồ: Admin_Portal nhận dữ liệu và vẽ biểu đồ lượt xem, giúp Owner theo dõi hiệu quả kinh doanh của chính mình một cách trực quan.

18. Kế hoạch Bảo trì & Hỗ trợ (Maintenance & Support)

- Sao lưu dữ liệu (Backup): Thiết lập lịch sao lưu tự động cho MongoDB định kỳ hàng ngày (Daily Backup).
- Logging: Sử dụng Serilog hoặc công cụ tương đương để ghi lại nhật ký hoạt động của hệ thống, giúp truy vết lỗi nhanh chóng khi có sự cố.
- Cập nhật Đa ngôn ngữ: Cung cấp giao diện cho Admin để cập nhật/thêm mới các bản dịch (Localization) mà không cần can thiệp vào mã nguồn (Code).
- Mở rộng: Kiến trúc Services/Controller tách biệt cho phép dễ dàng tích hợp thêm các tính năng như "Bình luận/Đánh giá" hoặc "Đặt chỗ" trong tương lai.